CANN 算子开发全链路体验升级

作者 **吴兆麟、袁江、刘斌**

时间 2026/03

目录

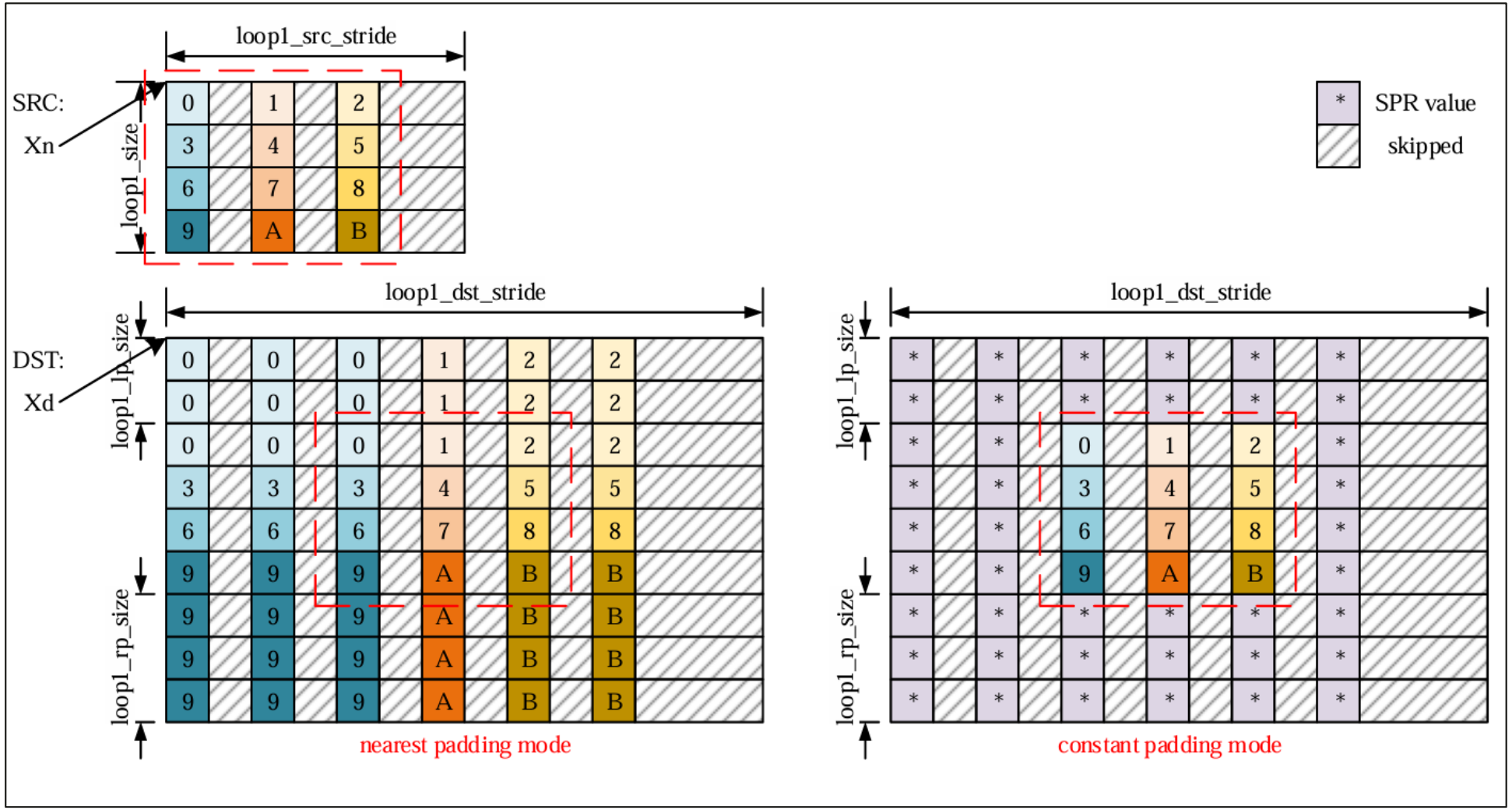
Part 1 算子编程易用性提升

Part 2 调优Profiling易用性提升

Part 3 运行时调度接口易用性提升

算子编程易用性提升-新增NDDMA能力(1)

新一代架构新增的 多层循环搬运 NDDMA 指令支持 GM→UB 的多维非连续访存、自动补 PAD，可高效实现转置、broadcast、as-strided、带多轴 PAD 等动态-shape 场景，显著提升数据搬运算子的易用性与性能。



CANN

算子编程易用性提升-新增NDDMA能力(2)

Ascend C对NDDMA的接口能力支持

```
template <typename T, uint8_t dim, const NdDmaConfig& config =
kDefaultNdDmaConfig>
__aicore__ inline void DataCopy(const LocalTensor<T>& dst, const
GlobalTensor<T>& src, const NdDmaParams<T, dim>& params);

// 模板参数配置
struct NdDmaConfig {
    static constexpr uint16_t unsetPad = 0xffff;
    // 1. 是否采用近似邻近值
    bool isNearestValueMode = false;
    // 2. 每个维度的Left Pad值
    uint16_t loopLpSize = unsetPad;
    // 3. 每个维度的Right Pad值
    uint16_t loopRpSize = unsetPad;
};

// 运行时参数配置
template <typename T, uint8_t dim>
struct NdDmaParams {
    NdDmaLoopInfo<dim> loopInfo;
    T constantValue;
};
```

Ascend C对NDDMA的接口能力支持

```
template <uint8_t dim>
struct NdDmaLoopInfo {
    // 表示每个维度内，该源操作数元素与下一个元素间的间隔
    uint64_t loopSrcStride[dim] = {0};
    // 示每个维度内，该目的操作数元素与下一个元素间的间隔
    uint32_t loopDstStride[dim] = {0};
    // 表示每个维度内，处理的元素个数（不包含Padding元素）
    uint32_t loopSize[dim] = {0};
    // 表示每个维度内，左侧需要补齐的元素个数
    uint8_t loopLpSize[dim] = {0};
    // 表示每个维度内，右侧需要补齐的元素个数
    uint8_t loopRpSize[dim] = {0};
}

// 在使用DataCopy接口进行数据搬运前，需要使用NdDmaDci接口刷新
__aicore__ inline void NdDmaDci();
```

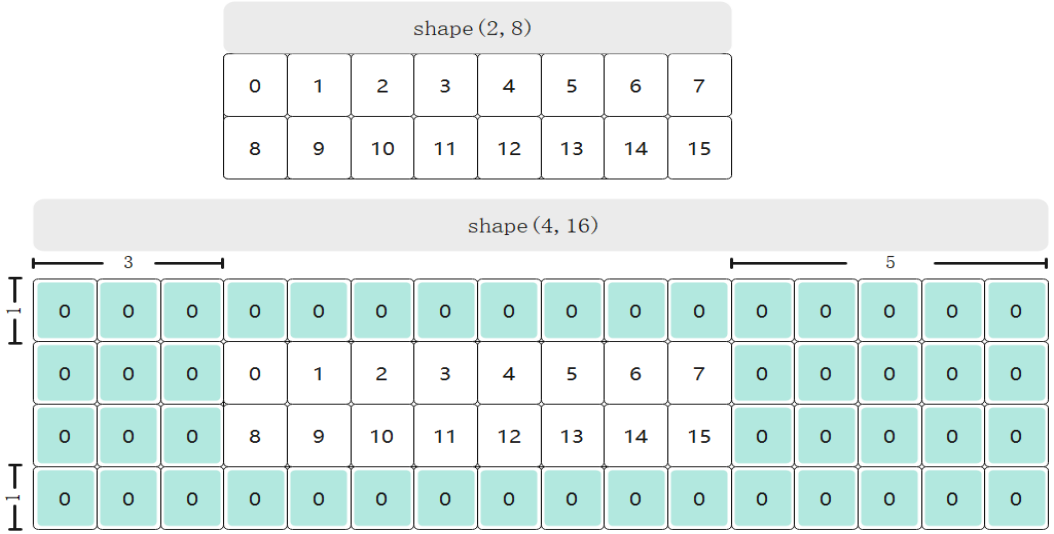
NDDMA接口定义

CANN

算子编程易用性提升-新增NDDMA能力(3)

2D Padding

```
// xGmShape：[2, 8]，搬运8列2行数据，左Padding 3，上Padding 1，右Padding 5，下Padding 1，xLocalShape：[4, 16]
AscendC::NdDmaLoopInfo<2> loopInfo{{1, 8}, {1, 16}, {8, 2}, {3, 1}, {5, 1}};
// padding的值为0
AscendC::NdDmaParams<T, 2> params{loopInfo, 0};
// 刷新cache
AscendC::NdDmaDci();
// 使用默认参数，也可以不传
static constexpr AscendC::NdDmaConfig dmaConfig;
AscendC::DataCopy<T, 2, dmaConfig>(xLocal, xGm, params);
```

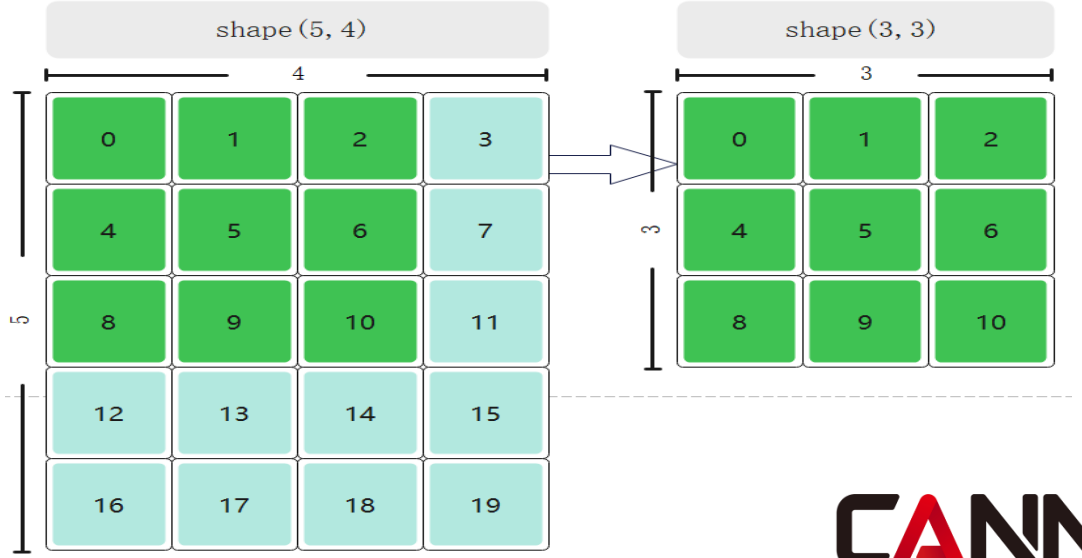


完整样例：NDDMA能力使能-2DPadding

Padding简化实现

2D Slice

```
// 2D BroadCast场景
// xGmShape：[5, 4] Slice 到ubShape[3, 3]
AscendC::NdDmaLoopInfo<2> loopInfo{{1, 4}, {1, 3}, {3, 3}, {0, 0}, {0, 0}};
// 设置info
AscendC::NdDmaParams<T, 2> params{loopInfo, 0};
// 刷新cache
AscendC::NdDmaDci();
// 使用默认参数，也可以不传
static constexpr AscendC::NdDmaConfig dmaConfig;
AscendC::DataCopy<T, 2, dmaConfig>(xLocal, xGm, params);
```

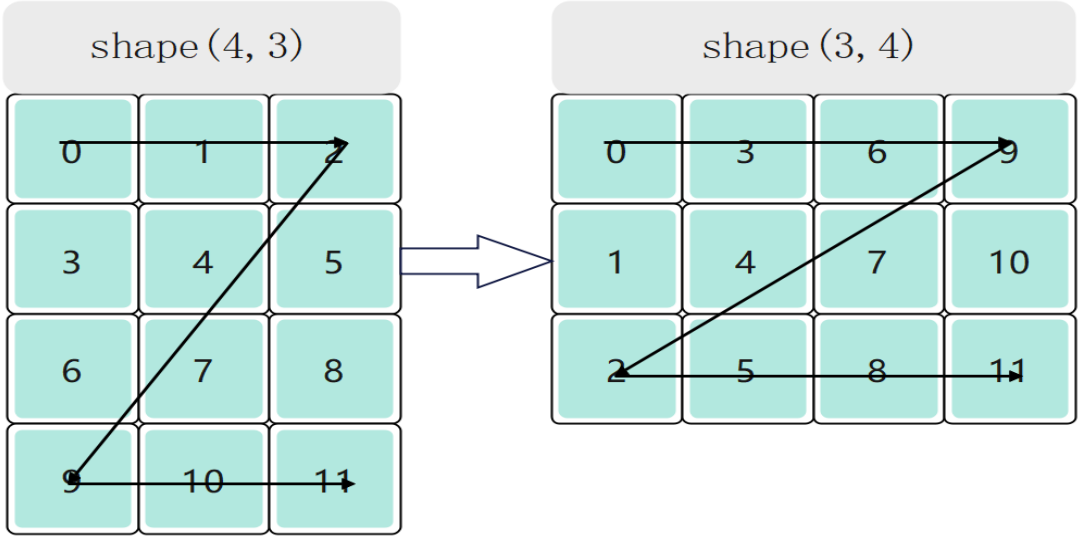


Slice简化实现

算子编程易用性提升-新增NDDMA能力(2)

2D Transpose

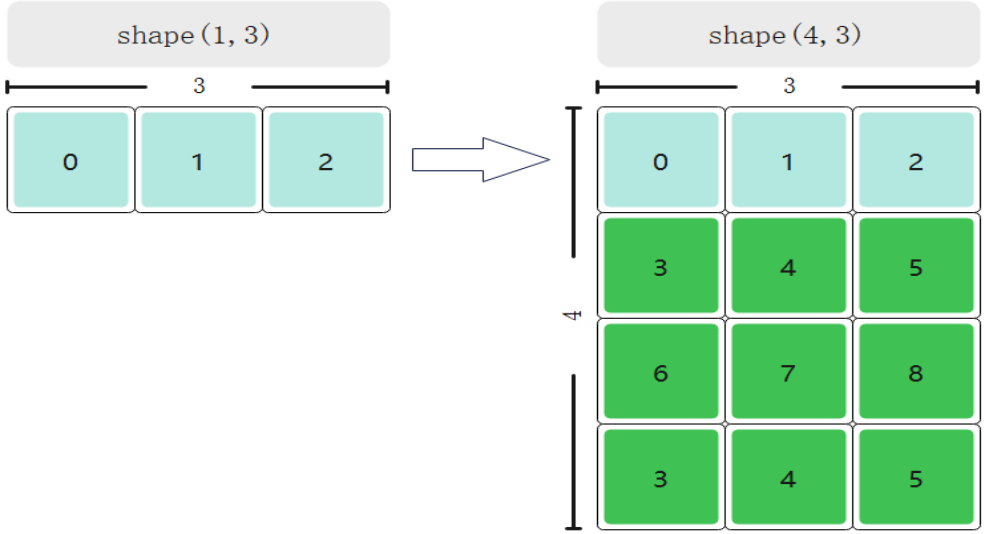
```
// 2D Transpose场景
// xGmShape：[4, 3] Transpose到 ubShape[3, 4]
AscendC::NdDmaLoopInfo<2> loopInfo{{1, 3}, {4, 1}, {3, 4}, {0, 0}, {0, 0}};
// 设置info
AscendC::NdDmaParams<T, 2> params{loopInfo, 0};
// 刷新cache
AscendC::NdDmaDci();
// 使用默认参数，也可以不传
static constexpr AscendC::NdDmaConfig dmaConfig;
AscendC::DataCopy<T, 2, dmaConfig>(xLocal, xGm, params);
```



Transpose简化实现

2D Broadcast

```
// 2D BroadCast场景
// xGmShape：[1, 3] Broadcast 到ubShape[4, 3]
AscendC::NdDmaLoopInfo<2> loopInfo{{1, 0}, {1, 3}, {3, 4}, {0, 0}, {0, 0}};
// 设置info
AscendC::NdDmaParams<T, 2> params{loopInfo, 0};
// 刷新cache
AscendC::NdDmaDci();
// 使用默认参数，也可以不传
static constexpr AscendC::NdDmaConfig dmaConfig;
AscendC::DataCopy<T, 2, dmaConfig>(xLocal, xGm, params);
```



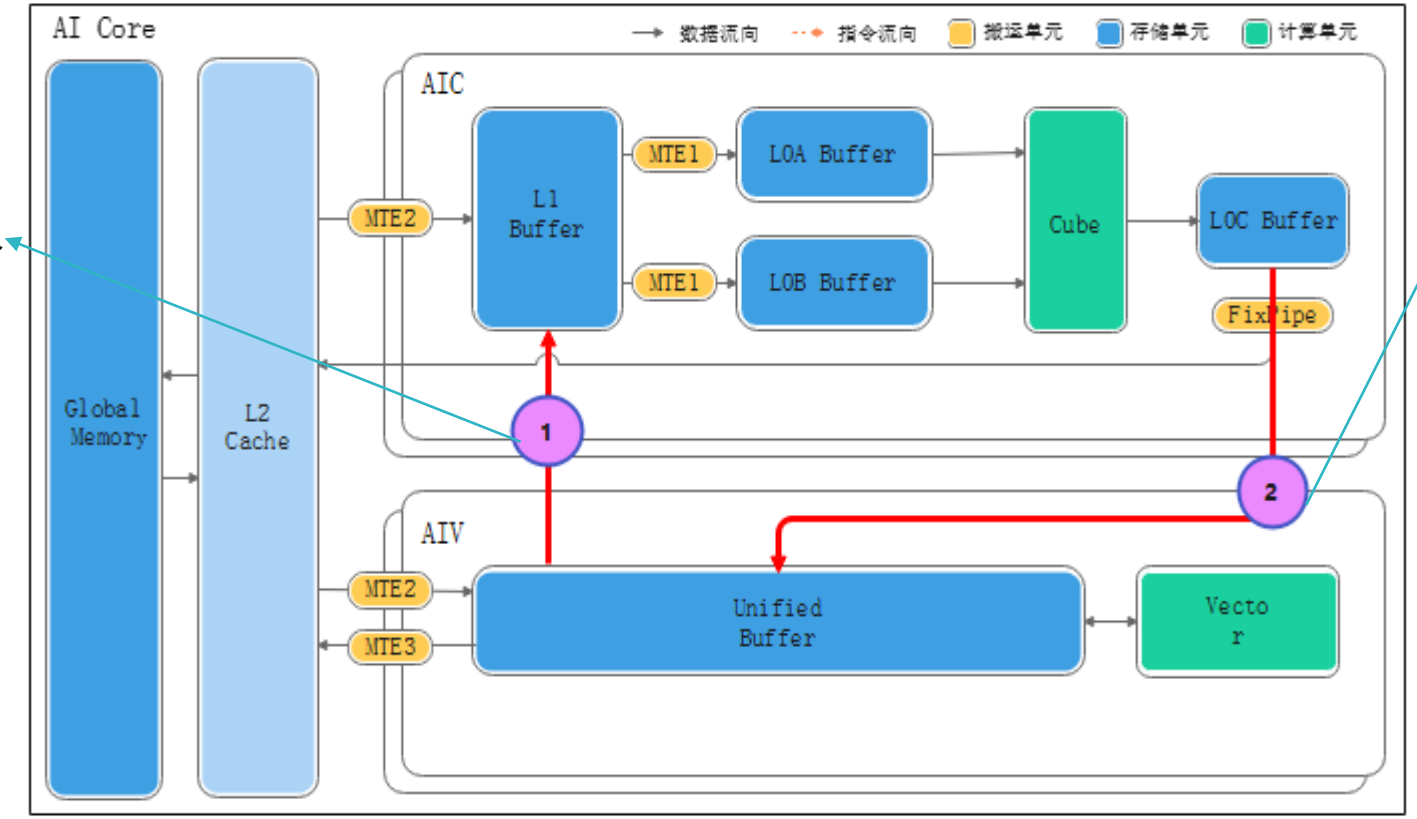
Broadcast简化实现

CANN

算子编程易用性提升-新增CV直连通路能力(1)

新架构引入UB2L1&L0C2UB间的直连通路，实现矩阵计算数据的快速搬移，旨在简化CV融合算子开发并提升性能。

矩阵搬入: 启用UB至L1（UB2L1）直连通路，通过DataCopy接口，支持融合算子的向量计算结果直接搬入L1。



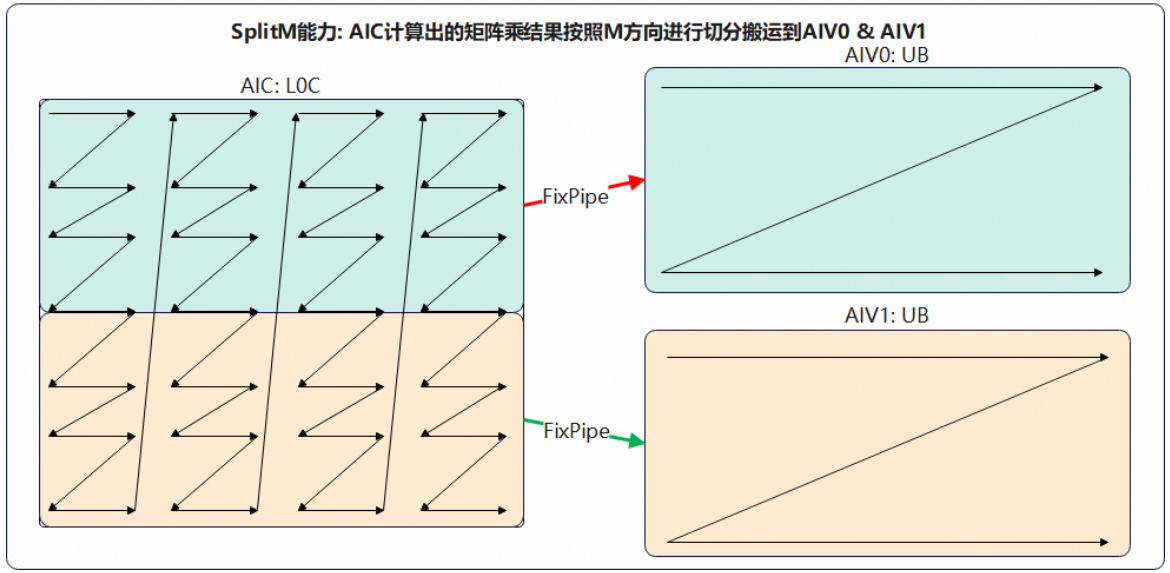
矩阵搬出: 启用L0C至UB（L0C2UB）直连通路，通过DataCopy接口，支持融合算子的矩阵计算结果直接搬入UB进行后续向量计算。

AI Core内 新增CV交互的通路

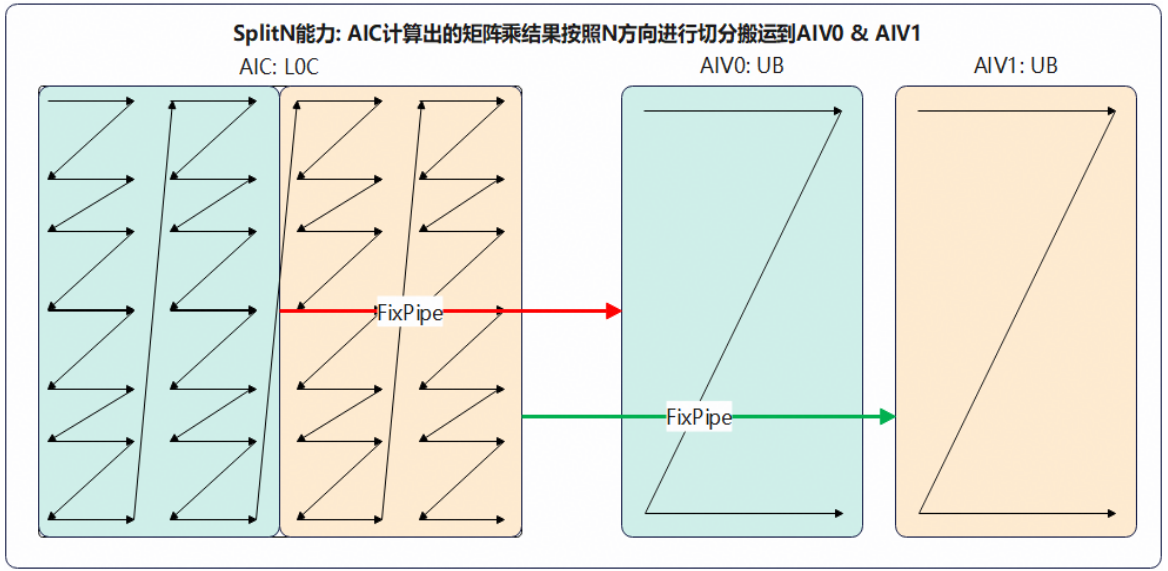
CANN

算子编程易用性提升-新增CV直连通路能力(2)

双目标模式控制，将L0C中的M×N矩阵将被分成两半，并同时写入两个AIV核的UB中。



- 按M维度拆分，M / 2 * N写入每个UB, M必须为2的倍数。



- 按N维度拆分，M * N/2写入每个UB, N必须为32的倍数。

CANN

算子编程易用性提升-新增CV直连通路能力(3)

关键使能接口定义

1. 新增: 搬入接口增加UB2L1的Nd2Nz搬入，支持Src&Dst都是LocalTensor的形式

```
template <typename T>
__aicore__ inline void DataCopy(const LocalTensor<T>& dst, const LocalTensor<T>& src, const Nd2NzParams& intriParams);
```

2. 新增: 搬出接口增加L0C2UB的搬出, 支持直接从L0C搬出到UB, 支持Src&Dst都是LocalTensor的形式

```
template <typename T, typename U, const FixpipeConfig& config = CFG_ROW_MAJOR>
__aicore__ inline void Fixpipe(const LocalTensor<T>& dst, const LocalTensor<U>& src, const FixpipeParamsC310<config.format>& intriParams);
template <CO2Layout format = CO2Layout::ROW_MAJOR>
struct FixpipeParamsC310 {
    // ...
    uint8_t dualDstCtl = 0;
};
```

3. 能力增强: 核间同步接口新增模式3

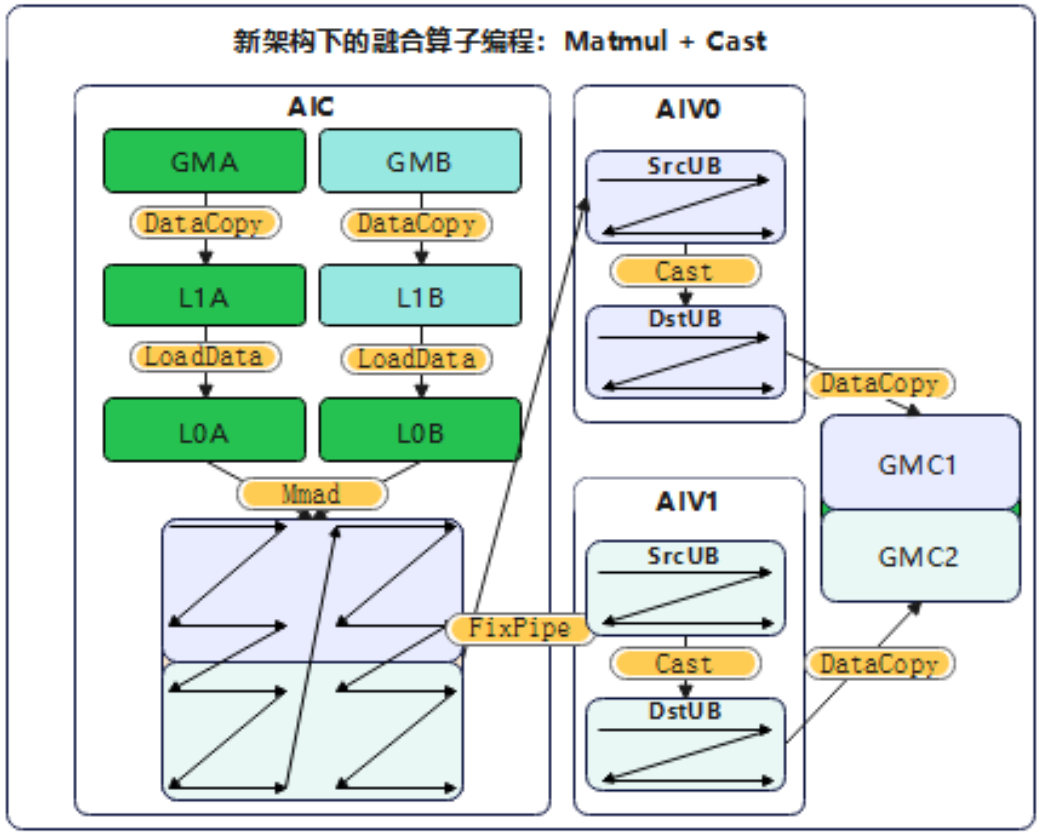
```
template <uint8_t modeId, pipe_t pipe>
__aicore__ inline void CrossCoreSetFlag(uint16_t flagId)

template <uint8_t modeId = 0, pipe_t pipe = PIPE_S>
__aicore__ inline void CrossCoreWaitFlag(uint16_t flagId)
```

mode增强: 新增模式3, AI Core内部，AIC与AIV之间的同步控制。AIV0与AIV1可单独触发AIC等待。
CrossCoreWaitFlag新增等待流水, 新架构支持配置除PIPE_S外的其他流水，能够更好的进行流水间的并行。

CANN

算子编程易用性提升-新增CV直连通路能力(4)

新架构新增的优化特性显著提升了CV流水并行效率，通过针对性优化流水同步机制，有效降低了同步开销，更有利于融合算子的性能优化。



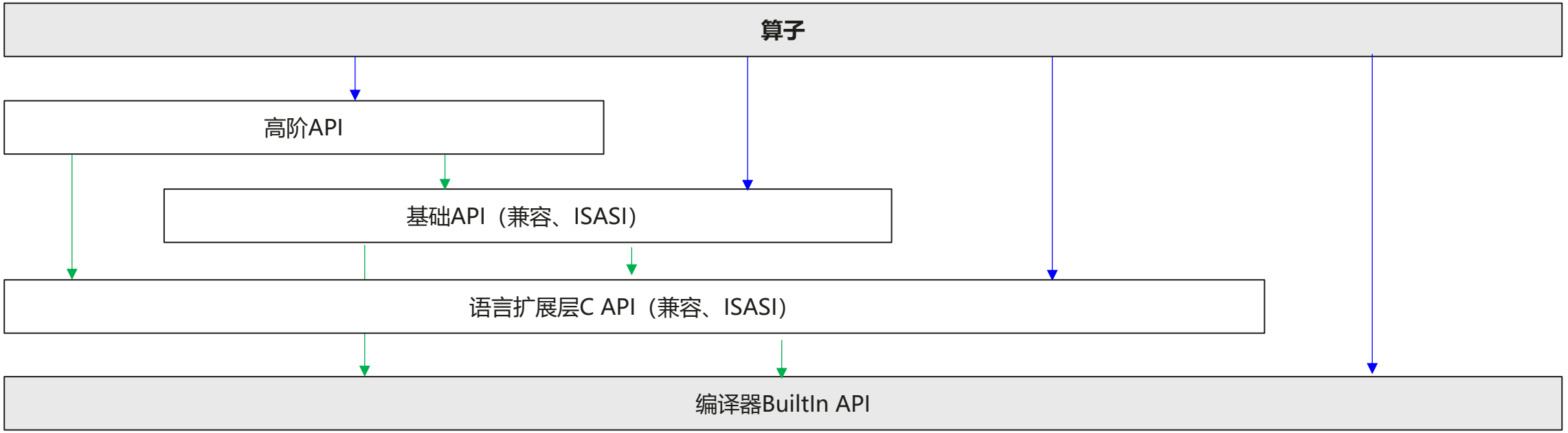

```
// AIC代码示例(关注计算流和核间同步，忽略核内同步)
__aicore__ inline void CubeOperator(const LocalTensor<DstT> &ubBuffer, bool transpose) {
    // ...
    // 1. 搬运和加载阶段忽略
    // ...
    // 3. 计算阶段: 使用MMad进行计算
    MmadParams mmadParams = { /*...*/ };
    Mmad(l0C, l0A, l0B, mmadParams);
    // 4. 搬出阶段：将计算结果搬出到UB中
    FixpipeParamsC310<CO2Layout::ROW_MAJOR> fixpipeParams(nLength, mLength, mLength,
nLength);
    fixpipeParams.params = { 1, 0, 0 };
    // 调用Fixpipe直接输出到UB中
    Fixpipe<DstT, L0cT, CFG_ROW_MAJOR>(ubBuffer, l0C, fixpipeParams);
    // 5. 通知Vector
    CrossCoreSetFlag<SYNC_MODE4, PIPE_FIX>(SYNC_AIV_AIC_FLAG);
}
```

```
// AIV代码示例(关注计算流和核间同步，忽略核内同步)
__aicore__ inline void CubeOperator(const LocalTensor<DstT> &ubBuffer, bool transpose) {
    // ...
    // 1. 等待Cube通知
    CrossCoreWaitFlag<SYNC_MODE4, PIPE_V>(SYNC_AIV_AIC_FLAG);
    // ...
    // 3. 使用UBBuffer进行计算处理
    // Vector计算
    Cast();
}
```

CANN

算子编程易用性提升-算子代际兼容处理(1)

Ascend C API层次结构



API层级	兼容策略目标
高阶API	高阶API保证同领域产品全平台兼容，特定领域扩展特性不兼容[1]
基础API	① 基础API区分兼容基础API和ISASI 基础API； ② 兼容API需要保证在所有产品上兼容； ③ ISASI API为体系架构相关API，对外不承诺跨芯片兼容， 如CUBE侧计算接口LoadData、MMAD等；
语言扩展层 C API	① C API可分兼容性/ISASI[2]
编译器BuiltInAPI	不保障兼容

注[1]：部分功能因兼容性代价较高，暂未完全支持兼容; [2]兼容性暂未支持，待规划

算子编程易用性提升-算子代际兼容处理(2)

硬件单元	910B/910C 到 950 的硬件能力变更	影响接口(基础API)	兼容迁移方案
搬运单元	删除L1Buffer到GM的数据通路	DataCopy/DumpTensor	1. cube only:在GM多分配一个单位矩阵，通过Mmad矩阵乘法计算输出到L0C Buffer，再从L0C Buffer通过Fixpipe搬运到GM 2. mix场景: 通过L1搬运到UB，再搬运到GM，需手动控制核间同步
	删除GM到L0ABuffer、L0BBuffer的数据通路	InitConstValue/LoadData	1. LoadData:实现GM-> L0A Buffer/L0B Buffer搬运需拆分成两步进行，先从GM搬运到L1 Buffer，再从L1 Buffer搬运到L0A Buffer/L0B Buffer 2. InitConstValue:先初始化L1 Buffer，再通过LoadData接口将将L1 Buffer上的数据搬运到L0A Buffer/L0B Buffer
计算单元	Cube计算单元不支持int4b_t类型	LoadData/LoadDataWithTranspose/Mmad	1. 算子侧通过MIX场景在vector core进行int4b_t到int8_t的转换，再通过UB搬运到L1后进行Mmad计算 2. 图层面可以在该算子前增加Cast节点进行int4b_t到int8_t的转换
	L0A分形变化ZZ->ZN	LoadData/LoadDataWithTranspose/Mmad	1. 非L0A Buffer切分的场景兼容当代版本 2. L0A Buffer切分的场景需要根据新的分形重新计算stride
	Vector Core Membase架构切换到Regbase架构	Membase基础API	1. 提供算子编译选项(-DNO_OVERLAP_IN_MULTI_REPEAT)优化性能(通过消除不必要的overlap插入的同步指令) 2. 极致性能可使用SIMD/SIMT VF编程
	不支持4:2稀疏矩阵的计算	LoadDataWithSparse/MmadWithSparse	直接使用稠密矩阵计算接口
存储单元	删除L1Buffer空间的边界值设定	SetLoadDataBoundary	1. 接口参数boundaryValue设置为0时（默认场景）功能兼容 2. 如果需要在L1 Buffer上循环读取操作数，需要将对应的LoadData(Load3D)接口手动拆分成多条指令，手动绕回
其他	删除UB异常调测接口	CheckLocalMemoryIA	该接口为调测接口，对功能无影响

注： 1. Matmul高阶API：不支持4:2稀疏特性；2. 当前不支持卷积计算类高阶API

CANN

算子编程易用性提升-算子代际兼容处理(3)

兼容案例一：修改Cmake中架构版本号直接编译实现兼容

```
target_compile_options(demo PRIVATE
    $<$<COMPILE_LANGUAGE:ASC>:--npu-arch=dav-2201>
    # NPU 架构号以正式发布的为准
    # $<$<COMPILE_LANGUAGE:ASC>:--npu-arch=dav-3510>
)
```

完整算子样例请参考：https://gitcode.com/cann/asc-devkit /asc-devkit/blob/master/examples/00_introduction/04_simple_operator/add_broadcast

兼容案例二：修改非兼容接口实现兼容

- 矩阵计算不再支持int4b_t
- 可通过Vector Core先将int4b_t转换成int8_t，然后再通过UB搬运到L1后进行Mmad计算

```
__aicore__ inline void Cast(...)
{
    // VectorCore Cast int4b_t 转换到 int8_t
    AscendC::Cast<half, AscendC::int4b_t>(tmpTensor, int4SrcLocalTensor,
AscendC::RoundMode::CAST_NONE, count * 2);
    AscendC::Cast<int8_t, half>(dstLocalTensor, tmpTensor,
AscendC::RoundMode::CAST_CEIL, count * 2);
    ...
}
```

完整算子样例请参考：https://gitcode.com/cann/asc-devkit /asc-devkit/blob/master/examples/04_best_practices/10_compatibility_cases/matmul_s4_950

CANN

目录

Part 1 算子编程易用性提升

Part 2 运行时调度接口易用性提升

Part 3 调优Profiling易用性提升

运行时接口优化：KernelLaunch调用优化

开发者加载和执行自定义算子，既可以通过<<<>>>方式，也可以通过Runtime接口。Runtime接口相比<<<>>>有更强大的扩展能力。

```
extern "C" __global__ __aicore__ void add_custom(GM_ADDR x,
GM_ADDR y, GM_ADDR z)
{
   KernelAdd op;
   op.Init(x, y, z);
   op.Process();
}
```

调用Runtime接口示例

```
 // 加载算子二进制
aclrtBinHandle bin;
aclrtBinaryLoadFromFile("add_custom.o", nullptr, &bin);

// 通过kernel名获取kernel句柄
aclrtFuncHandle add_custom;
aclrtBinaryGetFunction(bin, "add_custom", &add_custom);

// 申请算子所需的tensor内存
void *d_x, *d_y, *d_z;

// 下发算子任务
uint32_t numBlocks = 48;
void* args[] = {d_x, d_y, d_z};
size_t argsSize = 3 * sizeof(void*);
aclrtLaunchKernelWithHostArgs(add_custom, numBlocks, stream, nullptr, args, argsSize, nullptr, 0);
```

- 对于需要做核间同步的算子，Ascend 950相比上一代芯片无需在组装args时填写核间同步地址参数
- Ascend 950的核间同步算子的执行头开销相比上一代芯片从~3us优化到~150ns

CANN

运行时接口优化：KernelLaunch能力扩展

KernelLaunch接口可以通过aclrtLaunchKernelCfg参数实现更高的灵活性和可扩展性，该参数可以启用一些高阶特性。

填写kernellaunch扩展参数示例

```
// 下发算子任务

aclrtLaunchKernelCfg cfg;
aclrtLaunchKernelAttr attr[2];
attr[0].id = ACL_RT_LAUNCH_KERNEL_ATTR_SCHEM_MODE;
attr[0].value.schemMode = 1; // batch mode
attr[1].id = ACL_RT_LAUNCH_KERNEL_ATTR_TIMEOUT_US;
attr[1].value.timeoutUs.timeoutLow = 1000; // 最长执行时间1ms
attr[1].value.timeoutUs.timeoutHigh = 0;
cfg.numAttrs = 2;
cfg.attrs = attr;


aclrtLaunchKernelWithHostArgs(add_custom, numBlocks, stream,&cfg, args, argsSize, nullptr, 0);
```

- **ACL_RT_LAUNCH_KERNEL_ATTR_SCHEM_MODE：调度模式**

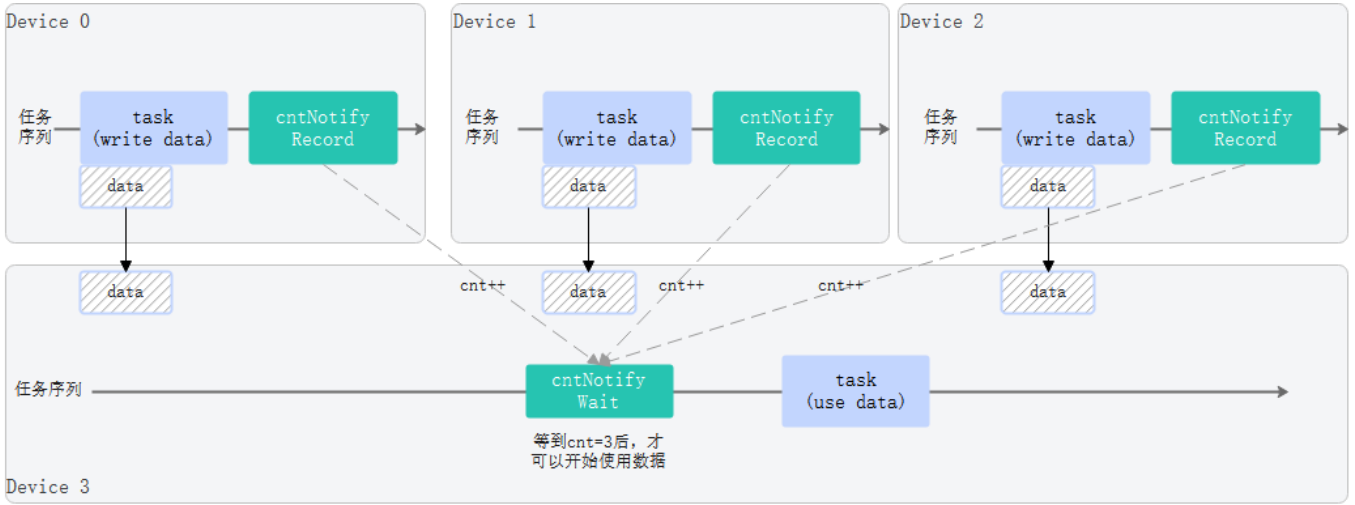
0：普通调度模式，只要有空闲的核，就启动算子执行。
1：batch调度模式，必须等numBlocks个核都空闲了，才启动算子执行。适用于需要全核同步的算子使用。

- **ACL_RT_LAUNCH_KERNEL_ATTR_TIMEOUT_US：算子超时时间，单位微秒**

如果算子执行时间超过配置的时间，则运行时会在流同步等接口返回超时错误。

CANN

运行时接口优化：新增Count Notify能力




```
// 创建count notify
aclrtCntNotify cntNotify;
aclrtCntNotifyCreate(&cntNotify, 0);

// 设置count notify为累加模式
aclrtCntNotifyRecordInfo info = {ACL_RT_CNT_NOTIFY_RECORD_ADD_MODE, 1};

// stream0将count notify计数+1
aclrtCntNotifyRecord(cntNotify, stream0, &info);

// stream1将count notify计数+1
aclrtCntNotifyRecord(cntNotify, stream1, &info);

// stream2将count notify计数+1
aclrtCntNotifyRecord(cntNotify, stream2, &info);

// stream3等待stream0/1/2的record都到齐
aclrtCntNotifyWaitInfo info = {ACL_RT_CNT_NOTIFY_WAIT_EQUAL_MODE, 3, 0, true, 0};
aclrtCntNotifyWaitWithTimeout(cntNotify, stream3, &info);
```

Count Notify提供丰富的模式，来简化1对N的同步编程

	模式	含义
record	赋值	Count = value
	累加	Count = 当前值 + value
	位或	Count = 当前值 | value
	位与	Count = 当前值 & value
wait	小于	如果Count < value，则解除wait
	等于	如果Count == value，则解除wait
	大于	如果Count > value，则解除wait
	大于等于	如果Count >= value，则解除wait
	掩码	如果Count & value == value，则解除wait

CANN

目录

Part 1 算子编程易用性提升

Part 2 运行时调度接口易用性提升

Part 3 调优Profiling易用性提升

调优：NPU性能调优流程

- PyTorch API
- TF API
- ACL API
- msprof
- 环境变量

msprof

- Timeline
- op summary
- mem
- 通信
- CPU
- …

msprof + Mindstudio Insight

- 核间负载图
- roofline
- Cache热力图
- Pipe流水图
- 算子代码热点图
- …

msprof op + Mindstudio Insight

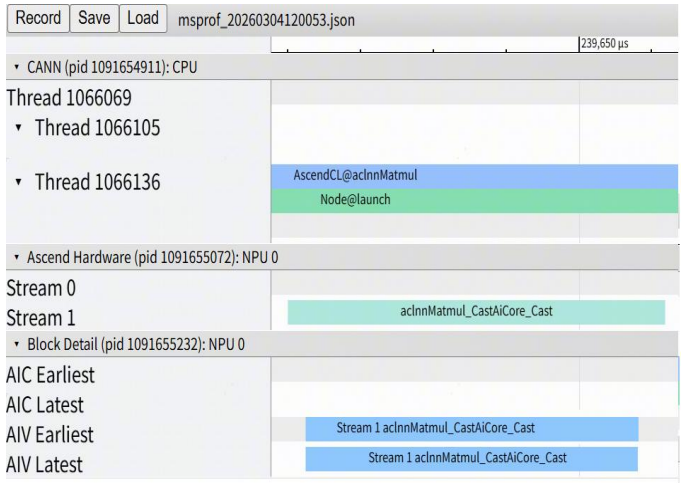
CANN

整网profiling-AICore算子首尾block profiling，算子调度&切分问题定界

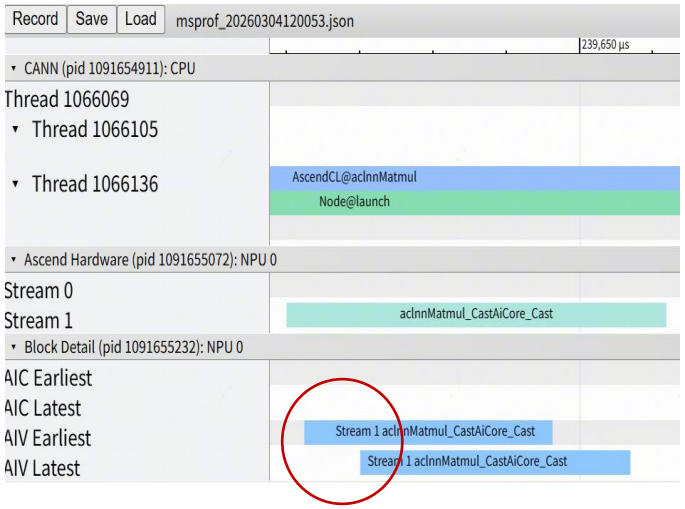
上一代

基于shape走读代码，判断此shape下block切分是否均匀，要求开发者要熟悉代码逻辑，效率较低

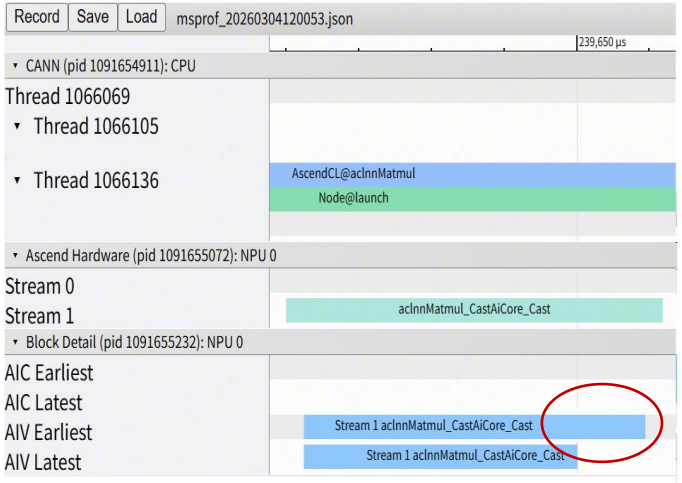
950



正常的AICore算子数据



AICore调度问题



算子block切分不均匀

支持开发者在整网调度中观察到第一个和最后一个调度的block性能数据，快速定界调度和算子切分的性能问题

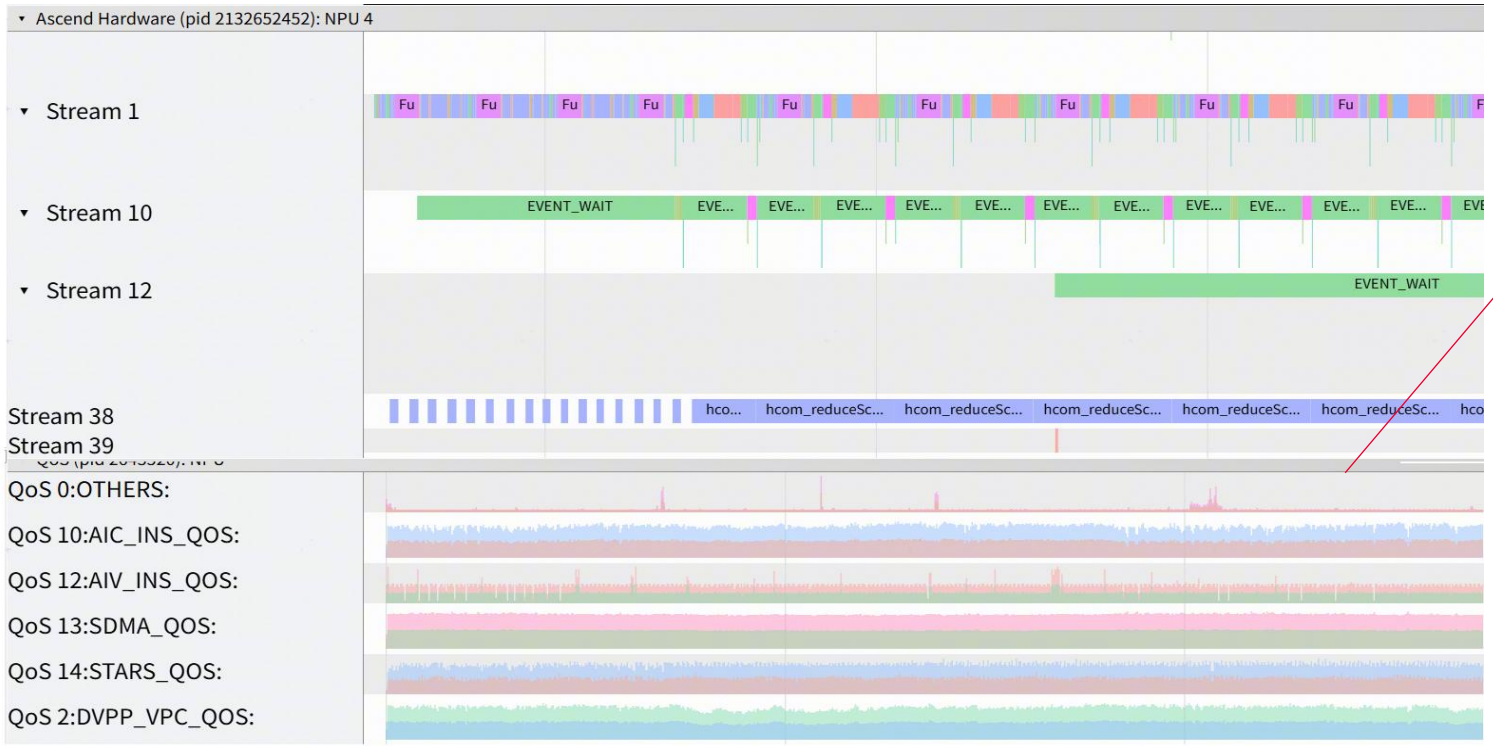
CANN

整网profiling-片上内存带宽，算子访存效率和抢占

上一代

片上内存带宽最快10ms采集一次，无法与算子的timeline做匹配，观察不到一个迭代内的片上内存带宽变化

950



默认场景下，没有QoS控制，profiling只会呈现QoS 0这一行
用户如果配置了加速器级别的QoS控制时，可以统计到不同加速器级别的带宽

采集频率从100hz提升到10000hz，开发者可以看到不同任务执行期间的带宽变化

CANN

整网profiling-CCU profiling，CCU内部关键性能打点

CCU是下一代硬件新增的集合通信专用的计算单元，可以有效降低集合通信对片上内存的访问压力，profiling增加了采集CCU性能数据的能力

Timeline展示CCU内部的关键指令执行耗时

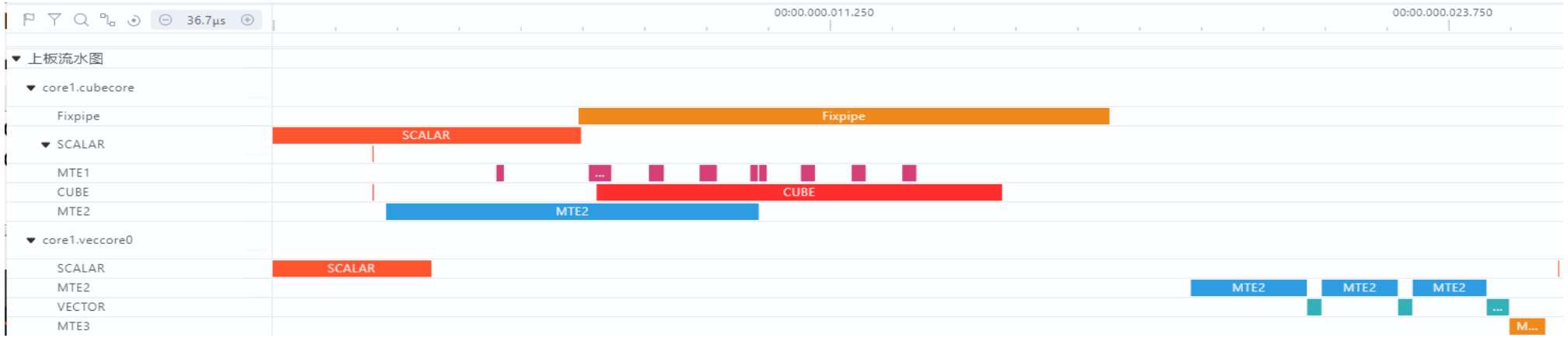
统计不同CCU任务各个channel的带宽

Device_ic	Channel Id	Timestamp(us)	Max Bandwidth(MB/s)	Min Bandwidth(MB/s)	Avg Bandwidth(MB/s)
0	0	1766558554812626.667	10803	5923	7134
0	1	1766558554812626.667	11909	6161	7013
0	2	1766558554812626.668	11791	6025	7211
0	3	1766558554812626.668	9901	6045	6832
0	4	1766558554812626.669	10602	5878	6678
0	5	1766558554812626.670	11959	6123	6903
0	6	1766558554812626.670	10521	6134	7103
0	7	1766558554812626.671	11853	6078	6843
0	8	1766558554812626.672	11531	5967	7212
0	9	1766558554812626.672	11732	5896	7321
0	10	1766558554812626.674	10943	6071	6843
0	11	1766558554812626.674	11326	6069	6934
0	12	1766558554812626.691	10831	6071	6912
0	13	1766558554812626.691	11943	5832	7045
0	14	1766558554812626.691	11762	5799	6832
0	15	1766558554812626.691	10723	6109	6789
0	16	1766558554812626.692	11901	5938	7041

CANN

算子profiling-Pipe流水图，快速识别算子内部流水bubble

通过上板执行采集AICore内部各个pipe的忙闲切换，识别算子内部计算和内存搬运流水bubble



通过AscendC::MarkStampImpl在算子代码中插入流水图打点，实现流水图与代码的关联

```
template <typename T>
__aicore__ inline void SimtComputeShell(GM_ADDR x
{
    __gm__ T* dst = (__gm__ T*)x;
    __gm__ T* src0 = (__gm__ T*)y;
    __gm__ T* src1 = (__gm__ T*)z;
    // VF_CALL启动SIMT VF子任务
        AscendC::MarkStampImpl<PIPE_V,14>();
    AscendC::Simt::VF_CALL<SimtCompute<T>>(AscendC
        AscendC::MarkStampImpl<PIPE_V,15>();
}
```

ID为14的打点

ID为15的打点

CANN

算子profiling-PC Sampling，定位SIMT算子阻塞代码和指令

场景：SIMT算子代码复杂，有较多分支，需要找到阻塞代码和指令
应用：采集硬件上报的指令阻塞状态并分析得到Stall Sampling表，让用户一眼找出阻塞比例最大的指令和状态

Tips：
支持悬浮展示9种Stall状态详情
详细展示各个阻塞状态采样次数
采样次数 ≠ 实际执行次数

#	Source	Inst...	Stall Sampling(All Sa...	Stall Sampling(Not I...
2	* Copyright (C) 2026. Huawei Technologies Co., Ltd. All rights reserved.			
3	*/			
4	#include "kernel_operator.h"			
5	using namespace AscendC;			
6				
7	template <typename T>			
8	__simt_vf__ __aicore__ LAUNCH_BOUND(2048) inline void SimtCompute(__gm__ T *dst, __gm__ T *src0, __gm_			
9	{	8	0.00%	0.00%
10	int idx = Simt::GetThreadIdx<0>();	48	0.00%	0.00%
11	int blockIdx = Simt::GetBlockIdx();			
12	int tmp = 10;			
13	for (int i = idx; i < idx + 2000; ++i) {	161...	0.01%	0.01%
14	auto val1 = src1[i];	161...	1.79%	1.70%
15	auto val2 = src0[i];	161...	21.47%	21.46%
16	auto val3 = val1 * val2 + tmp;	8093	53.81%	53.98%
17	if (val3 % 4 == 0) {	323...	0.02%	0.02%
18	val3 = val2 * tmp + val1;			
19	}			
20	dst[i] = val3;	242...	22.06%	21.99%
21	}			
22	}	15	0.00%	0.00%
23				
24	template <typename T>			
25	__aicore__ inline void SimtComputeShell(GM_ADDR x, GM_ADDR y, GM_ADDR z)			

指令关联代码

#	Address	Pipe	Source	Instruction...	GPR Count	GPR Status	Stall Sampling(All Sa...	Stall Sampling(Not Is...
2	0x1200443f70bc	VECTOR	VF_SIMT Xn, Xm, ...	4	0		0.00%	0.00%
3	0x1200443f70c4	SCALAR	MOVI	4	0		0.00%	0.00%
4	0x1200443f70c8	SCALAR	DCCI	4	0		0.00%	0.00%
5	0x1200443f70cc	SCALAR	END	4	0		0.00%	0.00%
6	0x1200443f70d0	VECTOR	S2R	8	2		0.00%	0.00%
7	0x1200443f70d8	VECTOR	S2R	8	3		0.00%	0.00%
8	0x1200443f70e0	VECTOR	SHFI	8	3		0.00%	0.00%
9	0x1200443f70e8	VECTOR	IADD.i	8	4		0.00%	0.00%
10	0x1200443f70f8	VECTOR	IADD.i	8	5		0.00%	0.00%
11	0x1200443f7108	VECTOR	START_DVG	8	3		0.00%	0.00%
12	0x1200443f7110	VECTOR	MOVI	8	4		0.00%	0.00%
13	0x1200443f7118	VECTOR	FMUL.i	8093	6		0.01%	0.01%
14	0x1200443f7128	VECTOR	FMUL.i	8093	8		0.01%	0.01%
15	0x1200443f7138	VECTOR	FMUL.i	8093	10		0.00%	0.00%
16	0x1200443f7148	VECTOR	LDG	8093	11		1.78%	1.70%
17	0x1200443f7150	VECTOR	LDG	8093	10		21.46%	21.46%
18	0x1200443f7158	VECTOR	FMUL.i	8093	9		53.81%	53.98%
19	0x1200443f7168	VECTOR	FMUL.i	8093	10		0.00%	0.00%
20	0x1200443f7178	VECTOR	Lop3	8093	9		0.00%	0.00%
21	0x1200443f7188	VECTOR	ISETP.i	8093	7		0.01%	0.01%
22	0x1200443f7198	VECTOR	SEL	8093	7		0.01%	0.01%
23	0x1200443f71a0	VECTOR	ISETP	8093	5		0.01%	0.01%
24	0x1200443f71a8	VECTOR	IADD.i	8093	4		0.01%	0.00%
25	0x1200443f71b8	VECTOR	STG	8093	3		0.81%	0.75%

Total: 73751
Active: 181
IBuf_Empty: 663
Nop_Cycles: 189
Others: 63
Register_bank_conflict: 84
Resource_conflict: 151
Scoreboard_Not_Ready: 72420

53.81% 53.98%

CANN

算子profiling-SIMT 寄存器分析-通过细粒度寄存器状态分析占用逻辑的合理性

#	Address	Pipe	Source	Instruction...	GPR Count	GPR Status	Stall Sampling(All Sa...	Stall Sampling(Not Is...
18	0x1200443f7158	VECTOR	FMUL.i	8093	9		53.81%	53.98%
19	0x1200443f7168	VECTOR	FMUL.i	8093	10		0.00%	0.00%
20	0x1200443f7178	VECTOR	Lop3	8093	9		0.00%	0.00%
21	0x1200443f7188	VECTOR	ISETP.i	8093	7		0.01%	0.01%
22	0x1200443f7198	VECTOR	SEL	8093	7	写入	0.01%	0.01%
23	0x1200443f71a0	VECTOR	ISETP	8093	5		0.01%	0.01%
24	0x1200443f71a8	VECTOR	IADD.i	8093	4	先读后写	0.01%	0.00%
25	0x1200443f71b8	VECTOR	STG	8093	3	读出	0.81%	0.75%

场景：需要针对SIMT算子的寄存器读/写使用情况进行细粒度分析。

应用：在SIMT算子运行过程中记录指令运行时的寄存器占用个数以及各寄存器状态

1. GPR Count：展示该指令执行时寄存器被占用的数量

2. GPR Status：展示各寄存器读写状态，用于分析寄存器占用逻辑是否合理。

CANN

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann

CANN